Git: The Complete Beginner to Pro Guide

What is Git?

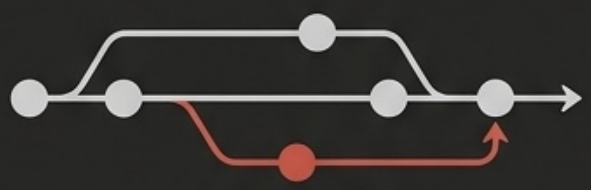
Git is a distributed version control system for tracking track changes in source code.

Why developers use Git

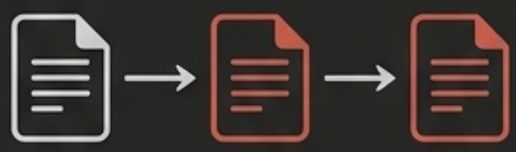
- Track changes
- Collaboration
- Manage versions

> *"Git is the backbone of modern software development."*

Version Control System explanation

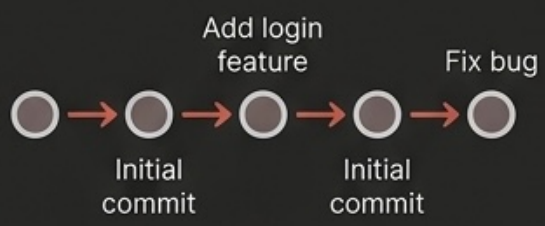


Version timeline



Git vs GitHub

Git	GitHub
Git is as local tool to control system	GitHub as the cloud hosting service



Benefits of Git

- Efficiency
- Data integrity
- Distributed workflow

Stuti Gujarathi
(codewithstuti)

Getting Started with Git

Install Git on:

Windows

- Install Windows staves on Windows instaumenis & Windows apr
- Check and have settings m installation.

Mac

- Install Git line Mac into Apple brew:betef installer
- Check out Ottvaetiitation from from and Mac

Linux

- Install Git line Linux Install-package installer
- Download brew and anf as dnf, llks packager penlapanaiog etc

Commands:

```
git --version
git version 2.39.1

git config --global user.name "Your Name"

git config --global user.email "email@example.com"
$
```

Check your Git version

Set your username and email globally

Set your username and email globally

Explain:

Global configuration

Global configuration is system-wide settings atrly-repredate to system and admicns ohdrirl configuration.

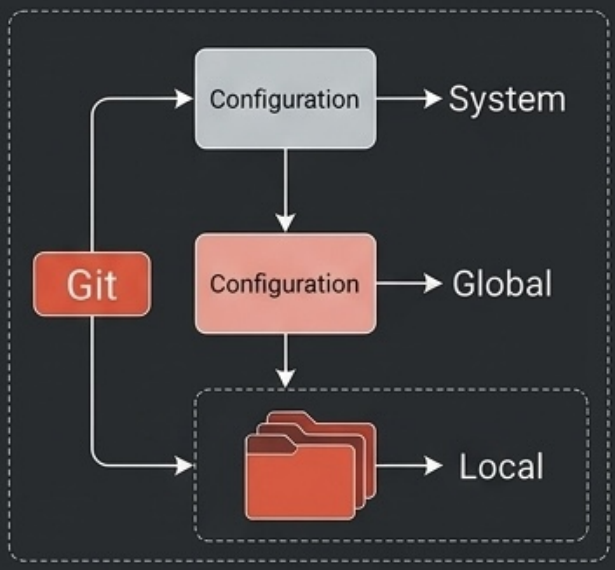
Local configuration

Single repository is oncle repository settingsrenrshand ot on a single repository in folders.

Checking Git settings

```
git config --list
```

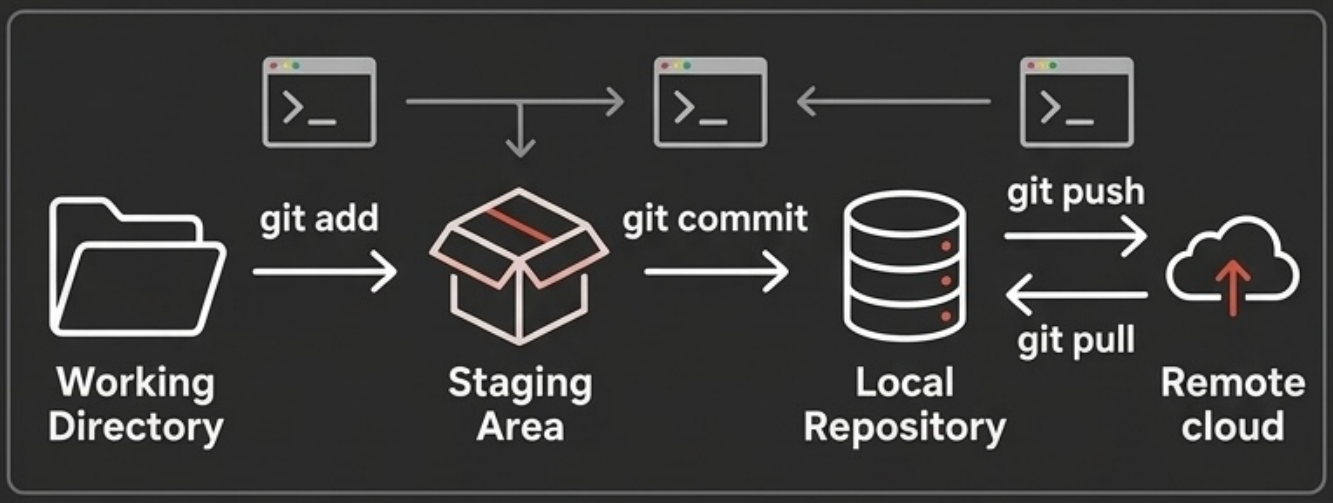
Checking recomimand configurng Git settings in the Local repository.



Setup workflow diagram

Stuti Gujarathi
(codewithstuti)

Essential Git Commands



git init

Initializes a new Git repository.

```
git init
```

git clone

Copies an existing remote repository.

```
git clone https://github.com/user/repo.git
```

git status

Shows the state of working directory and staging area.

```
git status
```

git add

Adds file changes to the staging area.

```
git add index.html (or git add .)
```

git commit

Saves a snapshot of staged changes to history.

```
git commit -m "Add login feature"
```

git push

Uploads local commits to a remote repository.

```
git push origin main
```

git pull

Fetches and integrates remote changes.

```
git pull origin main
```

Stuti Gujarathi
(codewithstuti)

Git Commands Quick Reference Guide

Git Checkout

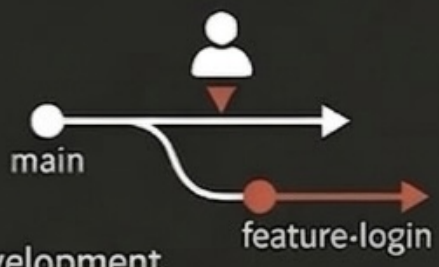
```
git checkout branch-name
```

Used to **switch branches** or **restore files.**

```
git checkout -b feature-login
```

(Create and switch to new branch)

Use case: Switch between features during development.



Git Restore

```
git restore filename
```

Used to **discard changes in files.**

Example use case: Revert accidental modifications in a file.

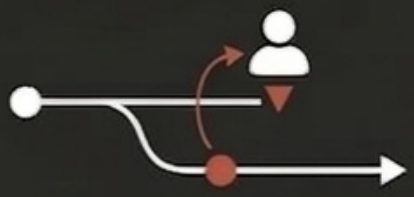
Git Switch Toggle

```
git switch branch-name
```

Modern command to change branches (simpler than checkout).

```
git switch -c feature-payment
```

(Create and switch branch)



Remote Sync

```
git fetch origin
```

Downloads changes from remote without merging.

Use case: See updates before merging them.

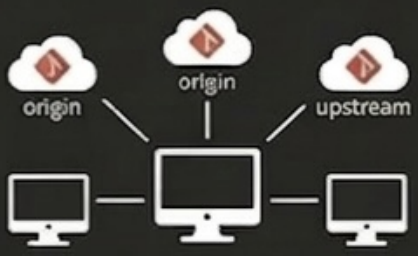
Git Remote

```
git remote -v
```

Used to manage remote repositories.

```
git remote add origin repository-url
```

Add remote repository

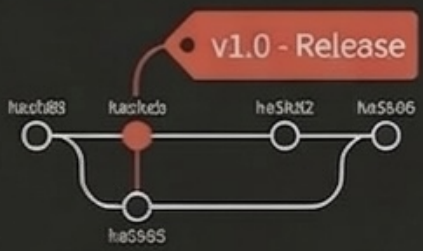


Git Tag

```
git tag v1.0
```

Used to mark important commits like releases.

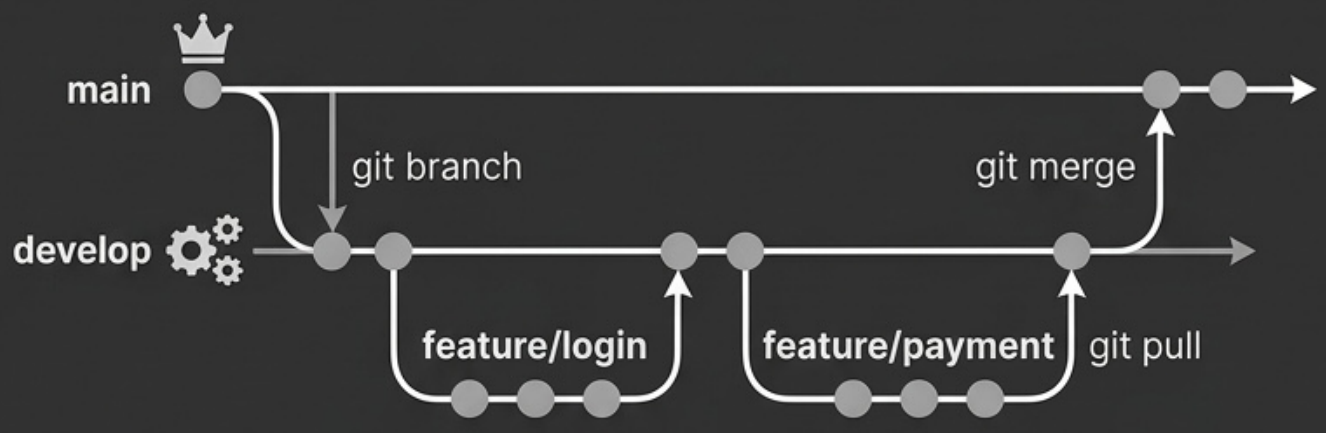
Use case: Mark software release versions.



Stuti Gujarathi
(codewithstuti)

Git Branching & Collaboration

Branching Strategy Visualized



`git branch`

Lists local branches or creates a new one.

```
git branch (or
git branch feature/signup)
```

`git switch` (and git checkout)

Switches to a specified branch.

```
git switch feature/signup
```

`git merge`

Integrates changes from one branch into another.

```
git merge feature/signup
```

`git rebase` (Advanced)

Re-applies commits to another base tip (caution advised).

```
git rebase main
```

Merge Conflicts Explained

Conflict in Code	Resolved

`sinc commit {`
`<<<<<<< HEAD`
`=======`
`>>>>>>> feature/greeting`
`char tap() {`
`  return "greeting";`
`}`
	`emar app() {`
	`char app();`
	`  return "/head";`
	`}`

Stuti Gujarathi
(codewithstuti)

Advanced Git for Developers

Git Stash

```
>_   git stash
```

Explanation
Save changes for maminento gits stash temporarily.

Use Cases

- Context switching without committing unfinished work

Git Log

```
commit: c7072e8502985f7537e258bb9453443d85378815e90f22e38c346538ac
Author: Seananen Aoner <uthor> <codewithstuti1j@gmail.com>
Date:
    View project commit history

hasheh: 5o685o7097c8565s38a237e7675359820367905546512 6Afaf5065176d
Author: Soenanen Aoner <athor> <sodewithstuti10@gmail.com>
Date:   Sun Dec, 01 Nar 2 39:SF:343 2022 +0000
    View project commit history

commit: 96519c8758503987357968955283293b836c25ed289127629827a29383
Author: Seananen Aoner <athor> <codewithstuti18@gmail.com>
Date:   Sun Nev, 01 Nar 2 18:03:153 2022 +0000

    User message
```

View project commit history

Use Cases

- Debugging and tracking changes

Git Tags

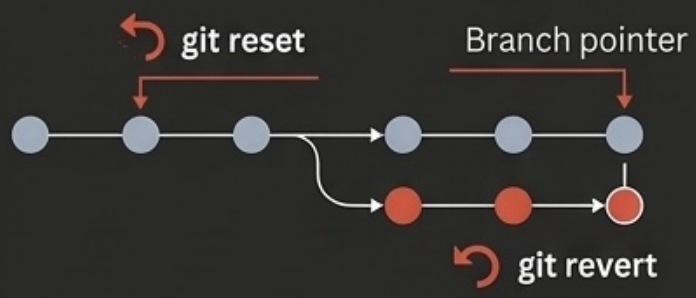
```
git tag <version>
```

```
git tag -a v1.0 -m "Release v1.0"
```

Bookmark important commits (releases)

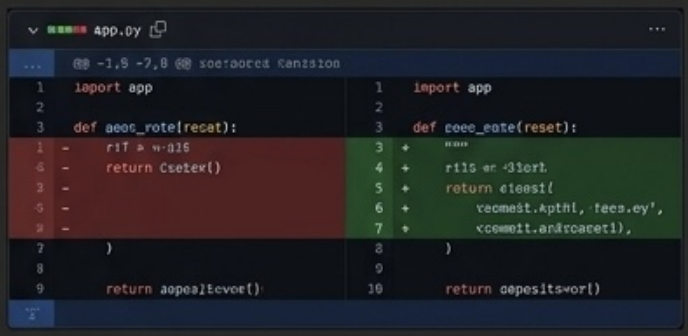
Use case: Version release management

Git Reset vs Revert



git reset (unstage/rewrite)	**git revert** (safe/public)
Moves the branch pointer back to same previous changes	Create a new commit changes: undoing previous changes

Git Diff




```
git diff
git diff <commit1> <commit2>
```

Git Ignore

```
# Ignore node_modules
node_modules/

# Ignore environment files
.env
```

Exclude specific files from being tracked

Use case: Keeping compiled files, logs, and secrets out of version control

Stuti Gujarathi
(codewithstuti)

Debugging Code with Git

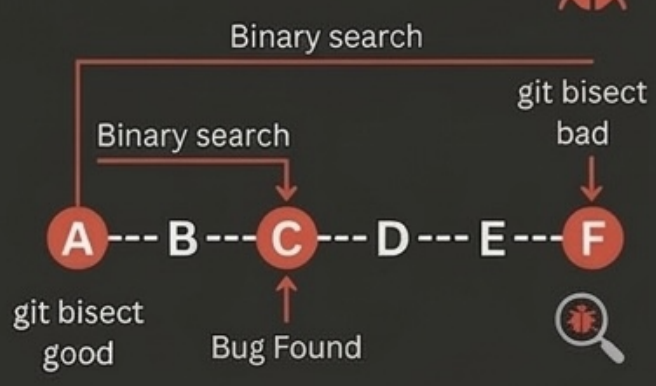
Git Bisect

Use binary search to find a bug-introducing commit.

```
git bisect start
git bisect bad
git bisect good <commit-id>
```

Workflow

1. Mark current commit as bad.
2. Mark older commit as good.
3. Git searches commits automatically.
4. Identify the bug-introducing commit.



Git Blame

Track code ownership, who wrote each line of code and when.

```
git blame filename
```

Example

```
a1b2c3d (John)  int calculateTotal()
d4e5f6g (Stuti) return price * quantity;
```

```
a1b2c3d (John)  int calculateTotal()
d4e5f6g (Stuti) return price * quantity;
```

Use cases

- Track code ownership
- Debug legacy code
- Understand history of a file

Stuti Gujarathi
(codewithstuti)

git | Advanced Repository Management

Git Reflog

Git Reflog records every change to the HEAD pointer, allowing developers to safely track HEAD history and recover lost commits.

Command

```
git reflog
```

Example terminal window with output:

```
abc123 HEAD@{0}: commit: added login feature
def456 HEAD@{1}: reset: moving to HEAD~1
```

Git command for commit recovery:

```
git checkout abc123
```

Use cases

- Recover deleted commits
- Undo resets
- Restore lost work

Git Submodules

Submodules allow a Git repository inside another repository.

```
git submodule add <repository-url>
```

Example use case

```
Main Project
 ├── frontend
 ├── backend
 └── 📦 shared-library (submodule)
```

Benefits

- Shared code across projects
- Modular development
- Independent repository management

'Stuti Gujarathi'
(codewithstuti)

Repository Optimization & Distribution

Git Archive

Git Archive creates a compressed snapshot of a repository without history.

```
git archive --format=zip HEAD > project.zip
```

Features and use cases

- Generates zip or tar files
- Used for deployment
- Shares project source without Git metadata

Git Garbage Collection (Git GC)

Git GC cleans and optimizes repository storage.

```
git gc
```

What it does

✔ Removes unreachable objects
✔ Compresses repository data
✔ Improves performance

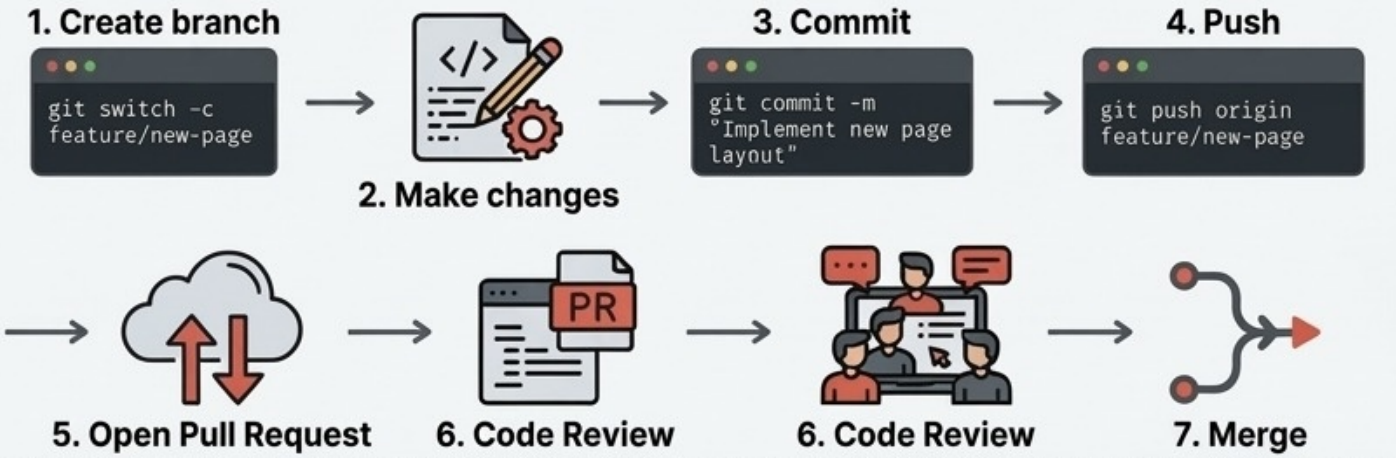
Git sometimes runs garbage collection **automatically** in background.

Stuti Gujarathi
(codewithstuti)

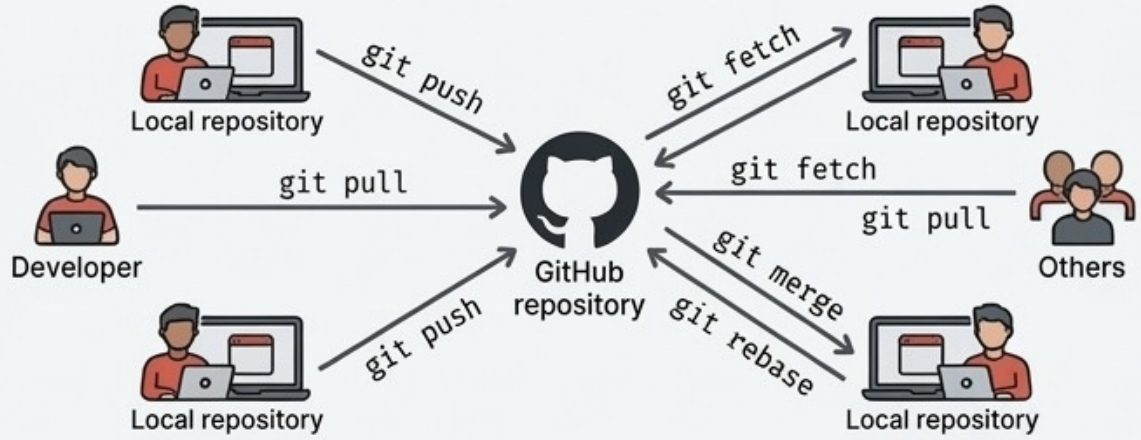
Professional Git Workflow

Workflows and workflows used in contemporary companies, including:

- **Feature Branch Workflow:** Isolates new features in dedicated branches, creating a separate branch with the dedicated branch name.
- **Git Flow:** A structured branching model with specific branch types at specified points (e.g., develop, main, feature, release, hotfix).
- **Pull Request Process:** Code changes are proposed, code is reviewed, approved, and integrated.
- **Code Review Workflow:** High importance is placed on peer code review for code quality, knowledge sharing, and overall verification.



GitHub collaboration diagram



Git Best Practices

- Write meaningful commit messages (e.g, 'Fix login bug' vs 'changes')
- Pull before pushing
- Use branches properly
- Use branches properly
- Keep commits small and focused
- Use .gitignore

"Good developers write code. Great developers manage code with Git."

Stuti Gujarathi
(codewithstuti)